INDIAN INSTITUTE OF TECHNOLOGY BHILAI

LABORATORY REPORT

Lab 8: Hands-on 1 to 4

CSL301: Operating Systems

Name	Rohit Raghuwanshi
ID	12341820
Course	Operating Systems (CSL301)
Date	6 October 2026

Hands-on 1: Toll Plaza Tag Reader

FIFO memory of K vehicle IDs: a found ID takes 1 second, a missing ID takes 5 seconds and is stored, and when memory is full the earliest stored ID is removed. The minimum number of slots is found by trying 1, 2, 3 ... in order.

Code: h1.c

```c
#include <stdio.h>

// FIFO simulation with k slots.
// Returns total time; mem[] holds the stored IDs (earliest first).
int simulate(int k, int n, int ids[], int mem[], int *count) {
    int size = 0, total = 0;

    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < size; j++) {
            if (mem[j] == ids[i]) {
                found = 1;
                break;
            }
        }

        if (found) {
            total += 1;                    // hit: order does not change
        } else {
            total += 5;                    // miss: ask the server
            if (size == k) {               // memory full: remove earliest
                for (int j = 0; j < size - 1; j++)
                    mem[j] = mem[j + 1];
                size--;
            }
            mem[size++] = ids[i];          // store the new ID at the end
        }
    }

    *count = size;
    return total;
}

int main(void) {
    int k, n, t;
    int ids[1000], mem[1001], tmp[1001];
    int count;

    scanf("%d", &k);
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &ids[i]);
    scanf("%d", &t);

    int total = simulate(k, n, ids, mem, &count);
    printf("Total time with %d slots: %d seconds\n", k, total);
    printf("Memory at end:");
    for (int i = 0; i < count; i++)
        printf(" %d", mem[i]);
    printf("\n");

    // Try 1, 2, 3 ... slots. More slots do not always help,
    // so we cannot binary search; check each one in order.
    int answer = -1;
    for (int s = 1; s <= n; s++) {
        int c;
        if (simulate(s, n, ids, tmp, &c) <= t) {
            answer = s;
            break;
        }
    }

    if (answer == -1)
        printf("Minimum slots for time <= %d: Not possible\n", t);
    else
        printf("Minimum slots for time <= %d: %d\n", t, answer);
    return 0;
}
```

Commands

```
$ clang -Wall -o h1 h1.c && ./h1 < h1_in.txt
$ ./h1 < h1_belady.txt                  # 3 slots (extra test)
$ printf '4\n12\n1 2 3 4 1 2 5 1 2 3 4 5\n48\n' | ./h1   # 4 slots (extra test)
```

Output

```
❯ printf '3\n7\n101 205 101 330 412 205 101\n25\n' > h1_in.txt
❯ clang -Wall -o h1 h1.c && ./h1 < h1_in.txt
Total time with 3 slots: 27 seconds
Memory at end: 330 412 101
Minimum slots for time <= 25: 4
❯ printf '3\n12\n1 2 3 4 1 2 5 1 2 3 4 5\n48\n' > h1_belady.txt
./h1 < h1_belady.txt
printf '4\n12\n1 2 3 4 1 2 5 1 2 3 4 5\n48\n' | ./h1
Total time with 3 slots: 48 seconds
Memory at end: 5 3 4
Minimum slots for time <= 48: 3
Total time with 4 slots: 52 seconds
Memory at end: 2 3 4 5
Minimum slots for time <= 48: 3
```

Output of h1: sample input (27 s, minimum slots 4) and the extra test

Think About

More slots do not always reduce the time. For the sequence 1 2 3 4 1 2 5 1 2 3 4 5, 3 slots take 48 seconds but 4 slots take 52 seconds (FIFO anomaly).

Hands-on 2: Phone RAM Manager

LRU with K apps in RAM: a warm start takes 1 second, a cold start takes 4 seconds and loads the app, and when RAM is full the app not opened for the longest time is killed. App names are compared with strcmp and copied with strcpy.

Code: h2.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    int k, n;
    char ram[100][32];      // index 0 = least recently opened
    char killed[100][32];
    int size = 0, kcount = 0, total = 0;
    char app[32];

    scanf("%d", &k);
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%31s", app);

        // look for the app in RAM
        int pos = -1;
        for (int j = 0; j < size; j++) {
            if (strcmp(ram[j], app) == 0) {
                pos = j;
                break;
            }
        }

        if (pos != -1) {
            total += 1;                          // warm start
            for (int j = pos; j < size - 1; j++) // move app to the end
                strcpy(ram[j], ram[j + 1]);
            strcpy(ram[size - 1], app);
        } else {
            total += 4;                          // cold start
            if (size == k) {                     // RAM full: kill ram[0]
                strcpy(killed[kcount++], ram[0]);
                for (int j = 0; j < size - 1; j++)
                    strcpy(ram[j], ram[j + 1]);
                size--;
            }
            strcpy(ram[size++], app);
        }
    }

    printf("Total time: %d seconds\n", total);

    printf("Apps killed:");
    if (kcount == 0)
        printf(" none");
    for (int i = 0; i < kcount; i++)
        printf(" %s", killed[i]);
    printf("\n");

    printf("Recent apps:");
    for (int i = size - 1; i >= 0; i--)     // most recent first
        printf(" %s", ram[i]);
    printf("\n");
    return 0;
}
```

Commands

```
$ clang -Wall -o h2 h2.c && ./h2 < h2_in.txt
$ printf '3\n3\nA B A\n' | ./h2          # no app killed (extra test)
```

Output

```
❯ printf '3\n7\nWhatsApp Chrome YouTube WhatsApp Maps Chrome WhatsApp\n' > h2_in.txt
❯ clang -Wall -o h2 h2.c && ./h2 < h2_in.txt
Total time: 22 seconds
Apps killed: Chrome YouTube
Recent apps: WhatsApp Chrome Maps
❯ printf '3\n3\nA B A\n' | ./h2
Total time: 9 seconds
Apps killed: none
Recent apps: A B
```

Output of h2: sample input and the "none killed" test

Hands-on 3: Mechanic's Tool Belt

Since the whole list is known in advance, on each trip the mechanic returns the tool whose next use is farthest away (the one placed earliest if several are never needed again). The program also counts the trips if the least recently used tool were returned instead.

Code: h3.c

```
#include <stdio.h>
#include <string.h>

int n, k;
char seq[100][32];      // tools needed, in order

// Position (after index i) where the tool is needed again; n if never.
int next_use(char *tool, int i) {
    for (int j = i + 1; j < n; j++)
        if (strcmp(seq[j], tool) == 0)
            return j;
    return n;
}

int find_tool(char belt[][32], int size, char *tool) {
    for (int j = 0; j < size; j++)
        if (strcmp(belt[j], tool) == 0)
            return j;
    return -1;
}

// Trips if the mechanic returns the least recently used tool.
int lru_trips(void) {
    char belt[100][32];
    int last_used[100];
    int size = 0, trips = 0;

    for (int i = 0; i < n; i++) {
        int pos = find_tool(belt, size, seq[i]);
        if (pos != -1) {
            last_used[pos] = i;
        } else {
            trips++;
            if (size < k) {
                strcpy(belt[size], seq[i]);
                last_used[size] = i;
                size++;
            } else {
                int old = 0;                        // smallest last_used
                for (int j = 1; j < size; j++)
                    if (last_used[j] < last_used[old])
                        old = j;
                strcpy(belt[old], seq[i]);
                last_used[old] = i;
            }
        }
    }
    return trips;
}

int main(void) {
    char belt[100][32];     // index 0 = placed on the belt earliest
    int size = 0, trips = 0;

    scanf("%d", &k);
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%31s", seq[i]);

    for (int i = 0; i < n; i++) {
        if (find_tool(belt, size, seq[i]) != -1)
            continue;                               // already on the belt

        trips++;
        if (size < k) {
            printf("Trip %d: take %s\n", trips, seq[i]);
        } else {
            // return the tool whose next use is farthest away;
            // strict ">" keeps the earliest-placed one on a tie
            int victim = 0;
            int far = next_use(belt[0], i);
```

```
            for (int j = 1; j < size; j++) {
                int nu = next_use(belt[j], i);
                if (nu > far) {
                    far = nu;
                    victim = j;
                }
            }
            printf("Trip %d: take %s, return %s\n", trips, seq[i], belt[victim]);

            for (int j = victim; j < size - 1; j++)    // remove victim
                strcpy(belt[j], belt[j + 1]);
            size--;
        }
        strcpy(belt[size++], seq[i]);              // new tool goes at the end
    }

    printf("Total trips: %d\n", trips);
    printf("Trips if returning least recently used tool: %d\n", lru_trips());
    return 0;
}
```

Commands

```
$ clang -Wall -o h3 h3.c && ./h3 < h3_in.txt
$ printf '2\n4\na b c d\n' | ./h3            # tie rule (extra test)
```

Output

```
❯ printf '2\n7\nhammer wrench drill hammer wrench saw hammer\n' > h3_in.txt
❯ clang -Wall -o h3 h3.c && ./h3 < h3_in.txt
Trip 1: take hammer
Trip 2: take wrench
Trip 3: take drill, return wrench
Trip 4: take wrench, return drill
Trip 5: take saw, return wrench
Total trips: 5
Trips if returning least recently used tool: 7
❯ printf '2\n4\na b c d\n' | ./h3
Trip 1: take a
Trip 2: take b
Trip 3: take c, return a
Trip 4: take d, return b
Total trips: 4
Trips if returning least recently used tool: 4
```

Output of h3: sample input and the tie-rule test

Think About

An operating system cannot know which pages will be needed in the future, but the mechanic has the full list for the day. This strategy needs that future knowledge, so real systems use past-based policies such as LRU.

Hands-on 4: Offline Music Storage

Both policies keep the stored songs in deletion order (front is deleted next). FIFO never reorders on a hit, LRU moves a hit song to the end. Songs are deleted from the front only until the new song fits; a song larger than C is only streamed.

Code: h4.c

```
#include <stdio.h>
#include <string.h>

int cap, s;
char name[100][32];
int size_of[100];

int find_song(char *nm) {
    for (int i = 0; i < s; i++)
        if (strcmp(name[i], nm) == 0)
            return i;
    return -1;
}

// policy: 1 = LRU, 0 = FIFO
// stored[] is kept in deletion order: stored[0] is deleted next.
//   FIFO: a hit does not change the order.
//   LRU : a hit moves the song to the end (most recently played).
void simulate(int policy, int p, int plays[],
              int *hits, int *misses, int *down, int stored[], int *count) {
    int used = 0, cnt = 0;
    *hits = *misses = *down = 0;

    for (int i = 0; i < p; i++) {
        int song = plays[i];

        int pos = -1;
        for (int j = 0; j < cnt; j++)
            if (stored[j] == song) {
                pos = j;
                break;
            }

        if (pos != -1) {
            (*hits)++;
            if (policy == 1) {                      // LRU: move to the end
                for (int j = pos; j < cnt - 1; j++)
                    stored[j] = stored[j + 1];
                stored[cnt - 1] = song;
            }
        } else {
            (*misses)++;
            if (size_of[song] > cap)                // too big: streamed
                continue;

            while (used + size_of[song] > cap) {    // delete only as needed
                used -= size_of[stored[0]];
                for (int j = 0; j < cnt - 1; j++)
                    stored[j] = stored[j + 1];
                cnt--;
            }
            stored[cnt++] = song;
            used += size_of[song];
            *down += size_of[song];
        }
    }
    *count = cnt;
}

void print_result(char *label, int hits, int misses, int down, int stored[], int count) {
    printf("%s: hits %d, misses %d, downloaded %d MB, stored:", label, hits, misses, down);
    for (int i = 0; i < count; i++)
        printf(" %s", name[stored[i]]);
    printf("\n");
}

int main(void) {
    int p, plays[100];
    char buf[32];

    scanf("%d", &cap);
```

```
    scanf("%d", &s);
    for (int i = 0; i < s; i++)
        scanf("%31s %d", name[i], &size_of[i]);
    scanf("%d", &p);
    for (int i = 0; i < p; i++) {
        scanf("%31s", buf);
        plays[i] = find_song(buf);
    }

    int h1, m1, d1, c1, h2, m2, d2, c2;
    int st1[100], st2[100];

    simulate(1, p, plays, &h1, &m1, &d1, st1, &c1);   // LRU
    simulate(0, p, plays, &h2, &m2, &d2, st2, &c2);   // FIFO

    print_result("LRU", h1, m1, d1, st1, c1);
    print_result("FIFO", h2, m2, d2, st2, c2);

    if (d1 < d2)
        printf("Better policy: LRU\n");
    else if (d2 < d1)
        printf("Better policy: FIFO\n");
    else
        printf("Better policy: Both equal\n");
    return 0;
}
```

Commands

```
$ clang -Wall -o h4 h4.c && ./h4 < h4_in.txt
$ printf '10\n4\nA 4\nB 4\nC 2\nD 6\n4\nA B C D\n' | ./h4   # several deletions
$ printf '10\n2\nA 4\nE 20\n3\nA E A\n' | ./h4              # oversized song
```

Output

```
❯ printf '10\n4\nA 4\nB 3\nC 2\nD 5\n7\nA B C A D A B\n' > h4_in.txt
❯ clang -Wall -o h4 h4.c && ./h4 < h4_in.txt
LRU: hits 2, misses 5, downloaded 17 MB, stored: A B
FIFO: hits 1, misses 6, downloaded 21 MB, stored: A B
Better policy: LRU
❯ printf '10\n4\nA 4\nB 4\nC 2\nD 6\n4\nA B C D\n' | ./h4
LRU: hits 0, misses 4, downloaded 16 MB, stored: C D
FIFO: hits 0, misses 4, downloaded 16 MB, stored: C D
Better policy: Both equal
❯ printf '10\n2\nA 4\nE 20\n3\nA E A\n' | ./h4
LRU: hits 1, misses 2, downloaded 4 MB, stored: A
FIFO: hits 1, misses 2, downloaded 4 MB, stored: A
Better policy: Both equal
```

Output of h4: sample input, several-deletions test and oversized-song test

Think About

In the test with songs A (4), B (4), C (2) and D (6) in 10 MB, adding D deletes only A and B and keeps C, so the program deletes exactly as many songs as needed.